코드잇 스프린트 AI엔지니어 부트캠프 15기

경구약제 이미지 객체 검출

프로젝트 결과보고서

탐지기와 분류기를 활용한 알약 위치 및 클래스 예측

4팀 - gitkk

팀원 : 김대현 · 김의연 · 진홍식

프로젝트 기간: 2026. 10. 22. – 2026. 11. 06.

제출일: 2026. 11. 05.

목차

1. 프로젝트 개요

1.1. 프로젝트 배경 및 목표
1.2. 과제 정의 및 접근 방법
1.3. 팀 구성 및 역할 분담
1.4. 프로젝트 기간 및 수행 과정

2. 데이터 분석

2.1. 데이터셋 구성 및 라벨 형식
2.2. 클래스별 데이터 분포
2.3. 이미지 및 바운딩 박스 특성 분석
2.4. 주요 발견 사항 및 모델링 고려 사항

3. 데이터 전처리 및 검증 설계

3.1. 데이터 정제 및 전처리
3.2. 데이터 증강
3.3. 학습·검증 데이터 분리
3.4. 평가 지표 및 검증 방법

4. 모델 설계 및 학습

4.1. 전체 예측 파이프라인
4.2. 탐지기 모델 구성 및 학습
4.3. 분류기 모델 구성 및 학습
4.4. 탐지·분류 결과 결합 및 후처리
4.5. 주요 학습 설정

5. 실험 및 성능 개선

5.1. 기준 모델 및 초기 성능
5.2. 주요 실험과 변경 사항
5.3. 실험별 성능 비교
5.4. 최종 모델 선정 근거

6. 최종 결과 및 분석

6.1. 최종 모델의 검증 성능
6.2. Kaggle 제출 결과
6.3. 예측 결과 시각화
6.4. 오류 사례 및 원인 분석

7. 협업 및 프로젝트 운영

7.1. 협업 흐름
7.2. Git 규칙 및 작업 양식
7.3. 핵심 코드 구조 및 실험 관리
7.4. 커뮤니케이션을 통한 문제 해결
7.5. 일정 및 역할 조정

8. 결론 및 회고

8.1. 프로젝트 성과 및 목표 달성도
8.2. 한계 및 개선 방향
8.3. 팀원별 회고

참고 자료 및 부록

참고 자료 및 출처
부록 A~E:
제출·재현·협업일지·검수·확인 사항

1. 프로젝트 개요

1.1. 프로젝트 배경 및 목표

본 프로젝트는 코드잇 스프린트 AI 엔지니어 부트캠프 15기 팀 프로젝트로, Kaggle의 「[AI15] 경구약제 이미지 객체 검출(Object Detection)」 프로젝트를 기반으로 수행하였다. 하나의 이미지에 포함된 최대 4개의 알약을 찾고, 각 알약의 종류와 위치를 클래스 및 바운딩 박스 형태로 예측하는 것을 목표로 하였다.

제공 학습 데이터는 232장·56종으로 제한되어 있었고, 테스트에는 학습 데이터에 없는 품목도 포함된다는 안내가 있었다. 이에 AI-Hub 데이터로 후보 품목과 표본을 보완하고, 데이터 분석·정제부터 모델 학습, 통합 평가와 Kaggle 제출까지 수행하였다.

1.2. 과제 정의 및 접근 방법

알약의 위치를 찾는 탐지기와 종류를 구분하는 분류기를 순차적으로 연결하였다. 탐지기는 모든 알약을 단일 pill 클래스로 처리하고, 분류기는 잘라낸 알약 이미지에서 118개 품목 중 하나를 예측하였다.

입력 이미지 → 알약 영역 탐지 → 영역 자르기 → 품목 분류 → 위치·품목·점수 출력

탐지기는 SSDLite320, Faster R-CNN, YOLO11s, YOLO26s를 검토하고 분류기는 ResNet-50을 사용하였다. 대회 평가 지표 mAP@[0.75:0.95]를 고려하여 높은 IoU 구간의 위치 정밀도를 중점적으로 평가하였다. 후보 품목의 범위와 모델 선정 근거는 2~5장에서 설명한다.

1.3. 팀 구성 및 역할 분담

프로젝트 참여자의 세부 역할은 아래에 정리하였다.

팀원	담당 업무
김대현	GitHub 저장소 구축 및 관리, 협업 규칙·개발 가이드 수립, 모델 설계, 코드 검토 및 리팩터링
김의연	탐색적 데이터 분석(EDA), 데이터 전처리, 모델 성능 개선을 위한 실험 수행 및 결과 검증
진홍식	탐색적 데이터 분석(EDA), 데이터 전처리, 프로젝트 일정 및 회의 진행

1.4. 프로젝트 기간 및 수행 과정

프로젝트는 2026년 10월 22일에 시작하여 11월 6일 발표 일정에 맞추어 진행하였다. 데이터 분석 및 전처리, 모델 학습, 예측 결과 검토와 Kaggle 제출을 수행하고, 프로젝트 과정과 결과를 보고서로 정리하였다.

프로젝트 시작: 10월 22일 · 보고서 및 저장소 완성: 11월 5일 19:00

발표: 11월 6일 · 협업일지 및 LMS 제출: 11월 6일 23:50

2. 데이터 분석

2.1. 데이터셋 구성 및 라벨 형식

Kaggle 데이터는 학습 이미지, 학습용 정답 어노테이션, 제출용 테스트 이미지로 구성되어 있었으며 별도의 검증 세트는 제공되지 않았다. AI-Hub에서는 대회 원천과 같은 촬영 조건의 조합 데이터를 확보하였다.

항목	**Kaggle** 대회 데이터	**AI-Hub** 원천 조합 데이터
규모	학습 232장·763박스·56종 / 테스트 842장	원천 PNG 14,012장: 촬영 사진 10,509장 + 보조 이미지 3,503장
제공 형태	학습 이미지·객체별 JSON / 테스트 정답 미제공	TL·TS 1, 3~8 사용, 사용 금지된 TL·TS 2 제외
해상도·형식	학습·테스트 1,074장 모두 976×1280 PNG(RGB)	976×1280 PNG; 연결 라벨 10,503장의 이미지 크기 정보 일치
촬영 조건	연회색 배경·주백색 조명; 70° 72장, 75° 75장, 90° 85장	라벨 10,503장 중 연회색 10,502장·검은색 1장; 주백색 조명, 각 각도 3,501장
라벨 정보	COCO 형식 bbox [x, y, w, h], 약 코드(category_id), 품목·외형·촬영 메타데이터	조합 폴더와 약 코드별 JSON; 원본 ID는 별도 매핑 필요

학습용 정답 라벨에는 각 알약의 종류와 위치가 기록되어 있었다. 한 이미지에 여러 알약이 포함되므로 클래스별 분포는 이미지 수가 아닌 알약 객체 수를 기준으로 분석하였다. 대회 학습 라벨의 알약 면은 앞면 751개·뒷면 12개였다.

파일명에는 약 코드 조합과 촬영 조건이 포함된다. 예를 들어 K-003351-016232-033880_0_2_0_2_75_000_200.png는 세 약품의 조합과 촬영 조건을 나타낸다.

위치	예시	의미·**JSON** 키
앞부분	K-003351-016232-033880	함께 찍힌 약 코드; 각 알약의 dl_mapping_code
1번째	0	정상알약 / drug_S
2번째	2	연회색 배경(0은 검은색) / back_color
3번째	0	앞면(1은 뒷면) / drug_dir
4번째	2	주백색 조명 / light_color
5번째	75	카메라 세로 각도 70·75·90° / camera_la
6번째	000	카메라 가로 각도 0 / camera_lo

7번째	200	조합 데이터 크기 값 / size

파일명 각 자리와 JSON 메타데이터의 관계는 약 40,000개 라벨을 대조하여 확인하였다. 같은 조합의 각도별 사진은 약 코드 앞부분이 같으므로 라벨 구성 검사와 그룹 분할에 활용하였다. 조합코드_index.png는 촬영 사진이 아닌 보조 이미지로 구분하였다.

2.2. 클래스별 데이터 분포

기존 학습 데이터는 클래스별 객체 수의 차이가 컸다.

항목	분석 결과
전체 클래스	56종
객체 수 10개 이하	29종
위 29종 중 객체 수 3개	17종
클래스별 최소 / 중앙값 / 최대	3개 / 9개 / 153개
최대·최소 비율	약 51배

AI-Hub 전체 약 5,000종을 품목당 30개씩 준비하면 약 15만 개의 표본이 필요해 준비·검수·학습 부담이 컸다. 대회 데이터가 AI-Hub TL·TS 조합 데이터에 기반한다는 정보를 고려하여 조합 데이터의 118종으로 후보 범위를 정하였다. 1차 선정된 AI-Hub 이미지에는 116종이 포함되어 있었으며, 나머지 2종은 Kaggle 학습 데이터로 보완하였다.

클래스 관계	수량
Kaggle·선정 AI-Hub 공통	54종
Kaggle에만 존재	2종
선정 AI-Hub에만 존재	62종
통합 고유 클래스	118종

Kaggle 테스트에는 학습 세트 밖의 품목이 포함된다는 안내가 있었고, 이후 운영진 답변에서 Public 정답의 품목 범위는 74종으로 확인하였다. 이는 테스트 전체 정답 분포를 직접 분석한 결과와는 구분하였다. 추가 선정·정제 후 데이터 규모와 클래스 분포는 3.1에 정리하였다.

2.3. 이미지 및 바운딩 박스 특성 분석

같은 조합을 각도만 바꾸어 촬영한 이미지가 많았다. 비슷한 구도라도 약품 종류·배치·앞뒤 방향이 달랐으므로 시각적 유사성과 동일 이미지 중복을 구분하였다. 1차 선정 이미지의 동일 픽셀 해시 중복은 없었지만, 이 검사만으로 학습·검증 간 유사성이나 누수 가능성이 모두 해소된 것은 아니었다.

분석 항목	관찰 내용
이미지당 알약 수	학습 라벨 기준 2개 7장·3개 151장·4개 74장
촬영 각도	70° 72장·75° 75장·90° 85장
박스 크기	가로·세로 중앙값 약 230×227px
라벨 오류	Roboflow 확인에서 대회 사진 14장에 박스 복사·위치·크기·누락 오류 관찰
유사 품목	에르도스테인 계열 캡슐, 흰색 원형 '20' 각인의 비유피-4정·리피로우정 등
학습·테스트 차이	측정상 밝기 차이가 있었고 선명도·알약 크기·위치는 유사함; 세부 수치는 3.2 참조

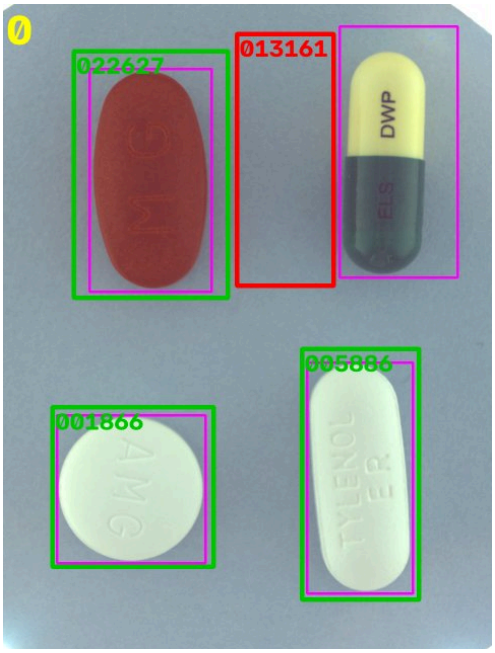


그림 2-1. 바운딩 박스 위치 오류 사례

빨간색 박스가 실제 캡슐 위치에서 왼쪽으로 벗어나 배경을 가리키고 있다. 빨강은 검수 대상 라벨 박스, 초록은 자동 검사에서 의심 후보로 분류되지 않은 라벨 박스, 분홍은 코드가 추정한 알약 영역이다.

이 사례는 모델의 탐지 실패가 아닌 데이터 라벨 검수 사례이다. 잘못된 박스는 분류용 crop 품질에도 영향을 줄 수 있어 원본 이미지와 라벨을 함께 확인하였다.

2.4. 주요 발견 사항 및 모델링 고려 사항

발견 사항	반영 방향
학습 표본과 후보 품목이 제한됨	AI-Hub 조합 데이터로 보강하고 118종 후보 구성
같은 조합의 각도별 사진이 많음	조합 단위 학습·검증 분리
클래스별 표본 수 차이가 큼	분류용 표본 선정과 역수 가중 샘플링
박스·품목 라벨 오류가 존재함	규칙 검사와 원본 확인을 결합한 2차 검수
높은 IoU에서 위치 정밀도가 중요함	엄격한 AP와 품목을 결합한 평가, crop 여백 검토
별도 검증 세트가 없음	내부 학습·검증 세트를 구성하고 외부 평가와 구분

이러한 관찰을 바탕으로 전처리와 검증 설계를 정하고, 데이터 변경 이력을 모델별 학습 패키지에 반영하였다.

3. 데이터 전처리 및 검증 설계

3.1. 데이터 정제 및 전처리

추가 데이터 선정과 클래스 매핑

원천 PNG 14,012장 중 보조 이미지와 연결·품질 조건을 충족하지 못한 데이터를 제외하고 연회색 배경·주백색 조명·70°·75°·90° 기준에 맞는 10,240장을 1차 선정하였다. 이미지와 JSON을 연결하고 바운딩 박스 좌표와 경계를 검사하였다.

원본과 대회 데이터의 식별 번호 차이는 K-코드 기준으로 매핑하였다. 원본 식별자는 보존하고 학습용으로 118종의 연속 ID를 구성하였다. 탐지기는 pill 1클래스로, 분류기는 개별 품목 라벨로 처리하였다.

단계	이미지	객체·crop	범위
최초 확보 합산	10,472장	40,155박스	대회 232장 + AI-Hub 1차 선정 10,240장
탐지기 v1	10,319장	39,588박스	학습 8,256장 / 검증 2,063장
탐지기 v2	10,169장	39,009박스	v1에서 150장·579박스 제외
분류기 v1	원본 4,048장	14,988 crop	학습 12,125 / 검증 2,863
분류기 v2	[(최종 원본 이미지 수) 보완 예정]	14,798 crop	학습 11,990 / 검증 2,808; 190 crop 제외

1차 검수와 제외 단위

라벨에 기록된 좌표, 크기, 약품 정보 등이 서로 맞는지 18개 항목으로 검사하였다. 검사 결과는 '명확한 오류', '사진을 보고 확인할 의심 사례', '데이터 분포의 치우침'으로 나누었다. 의심되거나 분포가 치우쳤다는 이유만으로 데이터를 제외하지는 않았다. 파일 연결, 좌표·면적·중첩, 약 코드·객체 수, 중복·유사성, 실물 크기 및 클래스별 분포를 확인하였다. 세부 기준과 검출 건수는 부록 D에 모았다.

탐지기는 사진 속 알약의 일부 박스만 제거하면 해당 알약이 배경으로 학습될 수 있어 이미지 단위로 제외하였다. 분류기는 수량이 적은 Kaggle 이미지의 정상 crop을 보존하기 위해 잘못된 박스만 제외하는 처리를 구분하였다.

대상	처리 단위	기록된 결과
분류기 AI-Hub 후보	오류 이미지 전체	후보 10,523장 중 393장 제외 — 범위는 M01 확인

분류기 Kaggle	잘못된 bbox	5장에서 7개 bbox 제외
탐지기 1차 선정 데이터	이미지 전체	AI-Hub 140장·Kaggle 18장 제외 후 v1 구성

분류용 표본 구성과 **2**차 검수

분류기는 품목당 108개 확보를 목표로 Kaggle 원본 232장과 조합 이미지 3,816장을 선정하였다. 품목별 수량을 정렬했을 때 108개 부근부터 분포가 이어지고 하위 7종과 차이가 있어 이를 목표로 정하였다. 최다 942개와의 비율은 약 8.7:1로 팀이 참고한 경험적 10:1 범위 안이었다. 정수계획(scipy milp)으로 표본 목표와 초과분을 고려하고 각 촬영 각도를 30% 이상 포함하며 서로 다른 조합을 우선하였다. 희소 품목의 가용 수량이 부족하거나 같은 이미지의 다른 약품이 함께 포함되어 모든 품목을 정확히 108개로 맞춘 것은 아니었다.

초기 분류기의 검증 정확도는 99.83%였으나 오답 5건을 원본과 비교하자 라벨 오류였다. 좌표 범위 검사만으로는 실제 알약과 박스의 일치나 품목 교환을 찾기 어려워, 픽셀 기반 위치 검사와 임베딩 기반 품목 검사를 추가하였다. 이미 개발된 외부 탐지기로 정답을 대체하지 않고, 자동 검사는 후보 선별에 사용한 뒤 사람이 원본을 확인하였다.

위치 검사는 Lab 배경 추정·Canny 윤곽선·형태학 연산으로 얻은 알약 영역을 정답 박스와 대응하였다. 품목 검사는 ResNet-50 임베딩을 이용한 다른 각도 간 비교와 조합 단위 5-fold 로지스틱 회귀로 후보를 찾았다. 알고리즘의 수치와 판정 규칙은 부록 D에 정리하였다.

검사	후보	오류	정상
박스 위치	135	52	83
품목 라벨	35	26	9

두 검사에 공통으로 잡힌 오류 19개를 제외하면 고유 오류는 59개였다. 위치 2순위 31건은 모두 정상으로 판정되어 자동 점수만으로 제외하지 않는 것이 중요하였다. 후보 기준 아래의 오류가 남을 수 있으며, 오류가 포함된 v1로 학습한 임베딩과 다른 각도 사진이 없는 crop의 비교에는 한계가 있었다.

AI-Hub 이미지 49장·188 crop 제외, Kaggle bbox 2개 제외, Kaggle 이미지 1장의 라벨 2개 교환·유지한다. kaggle의 경우 사용할 수 있는 데이터가 한정적이므로, 라벨을 교환해 유지가 가능한 경우 교환하여 활용하였다.

탐지기용 추가 검수에서는 기존 오류 이미지 48장과 추가 102장을 제외하여 v2를 구성하였다. 추가 의심 박스 330건을 확인해 오류 114건·정상 216건으로 판정하고 오류 포함 이미지 전체를 제외하였다. 남은 이미지의 분할·ID·정상 박스는 유지하였다.

모델별 입력과 가공 후 분포

탐지기는 원본 조합 이미지와 전체 박스를 사용하고, 분류기는 정답 박스 사방에 5% 여백을 주어 자른 뒤 정사각 패딩·224×224 변환을 적용하였다. 분할 목록·클래스 매핑·crop_pill 함수와 검수 기록을 공유하였다.

분포 지표	탐지기 **v1** 품목별 박스	분류기 **v1** 품목별 **crop**
최소 / 중앙값 / 최대	45 / 349 / 874	45 / 108 / 460
부족 품목	100개 미만 7종	108개 미만 8종
AI-Hub 출처 비중	이미지 약 98%	crop 95.0%

분류기 v1은 118종 모두 검증 표본이 있었으며 품목당 최소 9개였다. 희소 품목이 포함된 조합을 선택하면서 함께 찍힌 일부 품목은 456~460개로 목표를 넘었다. 최초 합산 단계의 최대 896개와 정제 후 v1 최대 874개는 집계 단계가 달라 구분하였다.

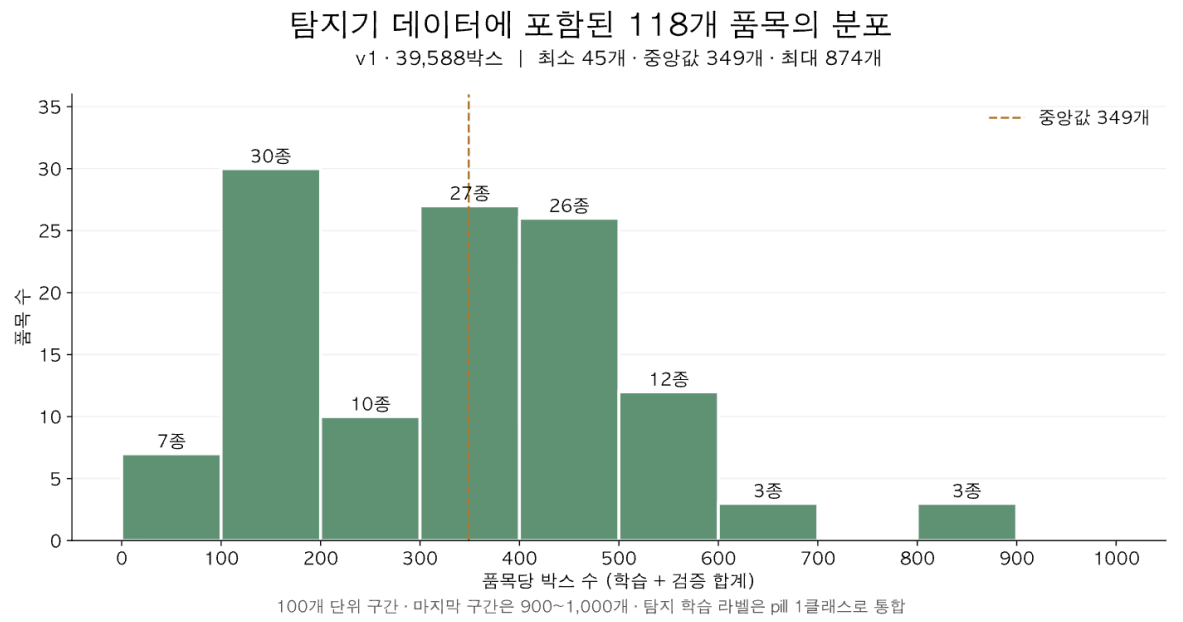


그림 3-1. 탐지기 v1 품목별 학습·검증 분포와 검증 배정 비율

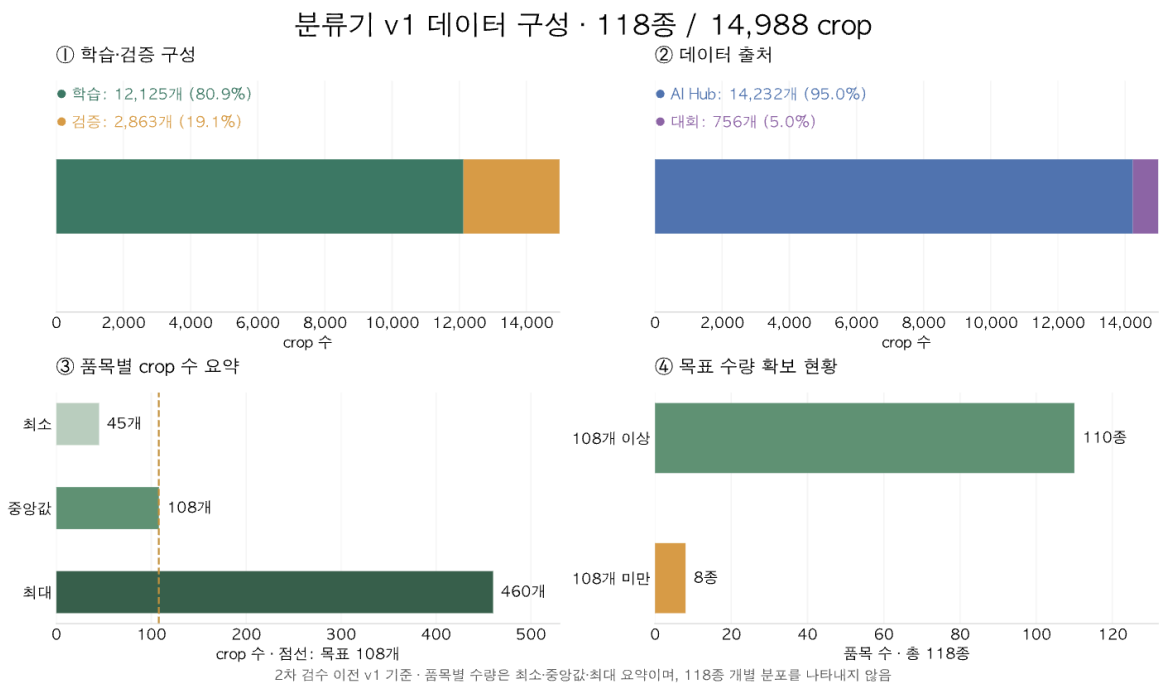


그림 3-2. 분류기 v1의 학습·검증 및 출처별 crop 구성

표본 수 차이는 클래스별 학습 수의 역수에 비례하는 WeightedRandomSampler로 보완하였다. 이는 새 이미지를 추가한 것이 아니다. 희소 7종의 부족분 207개를 단일 이미지로 채우는 방안은 zip 5개·413GB의 전체 다운로드가 필요하고 부분 다운로드가 불가능해 실행하지 않았다. 검수 전·후의 희소 품목 수는 단계별로 구분하였다.

3.2. 데이터 증강

품목을 구분하는 각인·색을 보존하고, 탐지 증강 시 이미지와 박스를 함께 변환하는 것을 원칙으로 하였다. 증강은 학습에만 적용하고 검증·추론에는 적용하지 않았다.

대상	적용 내용	선택 이유·범위
분류기	−180~180° 임의 회전; 캔버스 확장, RGB(124,116,104) 채움	방향 변화 대응; 뒤집기와 색 변환은 각인·색 단서 보존을 위해 제외
SSDLite320 기준선	증강 없음	v1·v2 기준선
SSDLite320 A1	좌우·상하 뒤집기 각 0.5, 밝기 ±0.15, 대비 ±0.2	관찰한 위치 편향·밝기 차이를 겨냥
Faster R-CNN	별도 증강 없음	기존 학습 자료 기준
YOLO26s	방향·색상·위치·크기 변화와 mosaic; 마지막 5 epoch mosaic 해제	YOLO11s 제출 이후에 고도화 작업

SSD A1 설계에서는 테스트 842장과 학습 이미지의 측정 결과를 참고하였다.

측정	테스트	학습	판단
밝기 중앙값	116	125	밝기 변화 시험
선명도 중앙값	38.4	36.2~36.4	흐림 증강을 추가할 근거 부족
알약 크기·위치	유사	유사	별도 크기·위치 변환 미적용
flip-TTA 위치 편향	—	박스 중심 약 1.2% 이동 관찰	뒤집기 학습으로 교정 검토

SSD A1에서는 임의 각도 회전으로 박스가 헐거워지거나 90° 회전·크기·위치 변환으로 기존 분포와 좌표 처리가 달라지는 위험을 고려하여 적용 범위를 제한하였다. 이 판단은 SSD A1에 대한 것이며 YOLO26s의 별도 증강 설정과 구분하였다. patience는 기준선 7에서 A1 10으로 조정하였다.

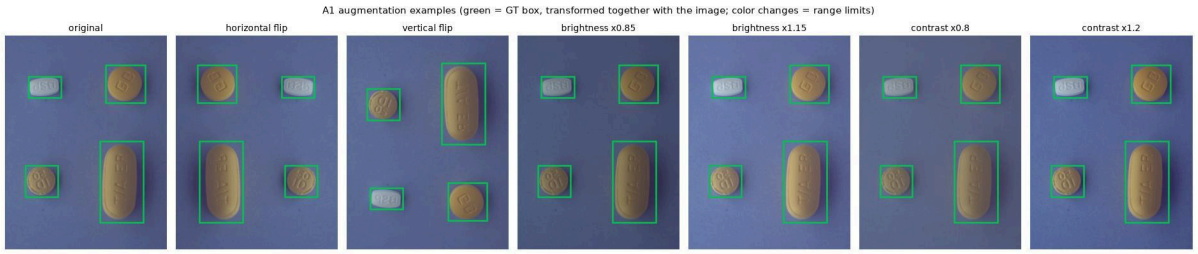


그림 3-3. SSD A1 뒤집기·밝기·대비 변환 예시

이미지와 박스는 apply_detector_augmentation에서 함께 변환하고 설정 범위를 학습 전에 검사하였다. 사전 채택 기준은 mAP@[0.75:0.95] 0.9502 이상과 상위 3개 epoch 평균의 동반 상승이었다. 실제 mAP는 0.9402→0.9498, AP@0.95는 0.7495→0.7844, 통합 오류는 12건→3건이었으나 채택 기준에 미달하고 Public도 0.58765→0.58430으로 하락하여 채택하지 않았다. 기준선 23 epoch와 A1 50 epoch의 차이 때문에 증강 효과와 학습 길이 효과를 분리할 수 없었다.

분류기 추가 증강·재학습을 보류한 이유와 정답 품목 대입 진단은 5.2 및 6.4에 정리하였다. 서로 다른 SSD 실행의 점수는 동일한 실험으로 합치지 않았다.

3.3. 학습·검증 데이터 분리

같은 알약 조합의 각도별 사진을 한 그룹으로 묶어 학습·검증을 약 8:2로 나누었다. 공통 그룹의 분할 불일치는 0개였으며, 분류기는 탐지기 분할을 따랐다. 탐지기 목록에 없는 분류기 9개 그룹·37 crop은 학습에 배정하였다.

데이터	학습	검증	합계
탐지기 v2 이미지	8,143장	2,026장	10,169장
탐지기 v2 박스	31,238개	7,771개	39,009개
분류기 v2 crop	11,990개	2,808개	14,798개

v1 기록상 품목별 검증 박스 비율은 평균 20.04%, 범위 13.26~26.67%였고 118종 모두 양쪽에 포함되었다. v2는 기존 분할을 유지하고 오류 표본을 제외하였다.

서로 다른 조합 간 유사 이미지까지 모두 제거한 것은 아니다. 분류기 v1 검증 crop 2,863개 중 799개가 학습 사진과 매우 유사한 것으로 확인되어 유사 사진을 제외한 평가도 검토하였다. 이는 정제 후 v2 검증 crop 2,808개와 분모가 다르다.

3.4. 평가 지표 및 검증 방법

모델 성능은 정답 crop 분류, 단일 pill 탐지, 품목까지 결합한 통합 평가로 나누었다. 분류기는 Top-1·macro F1, 탐지기는 mAP·recall, 최종 파이프라인은 category-aware mAP@[0.75:0.95]를 중심으로 평가하였다. IoU 0.50 기준 매칭은 오류 유형 분석에 사용하여 대회 지표와 구분하였다.

공통 탐지기 비교에는 동일한 ResNet-50 분류기와 탐지기 v2 검증 이미지 2,026장을 사용하였다. 별도로 학습에 사용하지 않은 AI-Hub Validation 조합 사진 1,500장·5,520박스·54종을 분류기 평가 전용 세트로 구성하였으며, resnet50_baseline_v2의 macro F1은 0.9981이었다.

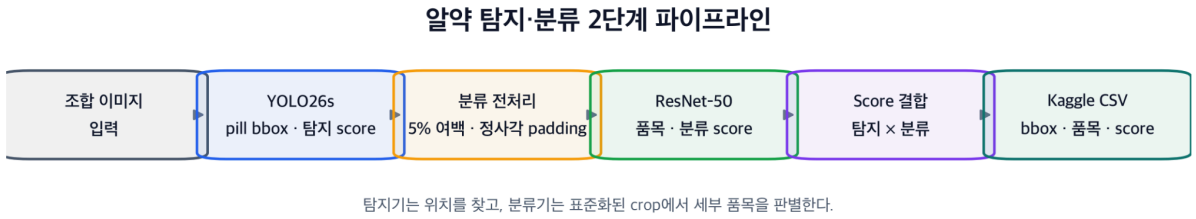
Public 정답에 포함된 74종의 AP 평균이라는 채점 방식은 운영진 답변으로 확인하였다(부록 F에 문의 및 답변 내용 첨부). 평가 분모에 대한 초기 가설과 제출 진단은 5.3·6.2 및 부록 A에서 설명한다.

4. 모델 설계 및 학습

4.1. 전체 예측 파이프라인

본 시스템은 한 장의 조합 이미지에서 알약의 위치를 찾는 탐지 단계와, 각 bbox를 세부 품목으로 구분하는 분류 단계를 순차적으로 연결하였다. 탐지기는 모든 알약을 하나의 pill 클래스로 처리하고 분류기는 현재 데이터에 정의된 품목 중 하나를 예측한다. 품목 구성이 바뀌더라도 탐지기는 유지하고 분류기만 새로운 품목 구성에 맞춰 재학습할 수 있다.

그림 4-1. 알약 탐지·분류 2단계 파이프라인



* 조합 이미지에서 알약을 탐지하고, *bbox* 전처리와 품목 분류를 거쳐 *Kaggle* 제출 결과를 생성하는 전체 흐름이다.

탐지와 분류를 별도 모델로 구성한 이유는 두 작업의 책임을 명확히 나누기 위해서다. 탐지기는 품목 수와 관계없이 위치 탐지에 집중하고, 분류기는 표준화된 crop에서 색상·형태·각인 차이를 학습한다. 동일한 분류기를 여러 탐지기와 연결할 수 있어 탐지 모델 비교가 분류 모델 변경의 영향을 받지 않는다. 여기서 2단계 파이프라인은 탐지기와 별도 분류기를 순차적으로 연결한 구성이며, Faster R-CNN 내부의 RPN·RoI head로 이루어진 two-stage 탐지 구조와는 구분된다.

초기에는 분류기와 탐지기의 입력 metadata와 실행 방식이 달라 통합 검증이 어려웠다. 이에 입력 명세와 검증 기준, Notebook 실행 순서를 통일하고 탐지·분류 결합 로직을 pipeline으로 분리해 하나의 실행 흐름으로 정리하였다.

탐지기와 분류기에는 원본 조합 이미지 단위로 동일하게 분할한 train·validation 데이터를 사용하였다. 이를 통해 같은 알약 조합의 각도별 이미지가 두 데이터에 나뉘어 들어가는 것을 방지하였다.

추가 설계 근거로 분류와 박스 회귀의 역할을 분리하는 Double-Head R-CNN(CVPR 2020)을 참고하였다. 이를 본 프로젝트의 별도 탐지기·분류기 구조와 동일한 방식으로 간주하지는 않았으며, 세부 품목의 각인·색상 차이를 원본 해상도 crop에서 학습하고자 하였다. 특히 같은 '20' 각인의 흰색 원형 알약처럼 외형이 비슷한 품목을 고려하였다. 이 구조는 추론을 두 번 수행하고 데이터 패키지 두 종류를 관리해야 하며, 탐지 위치 오차가 분류 입력에 전파되는 비용이 있었다.

4.2. 탐지기 모델 구성 및 학습

탐지기는 원본 조합 이미지에서 모든 알약의 bbox와 객체 score를 예측하는 단일 클래스 모델이다. 세부 품목은 후속 분류기가 판별하므로 탐지 단계에서는 모든 알약을 pill로 통합하였다. 탐지 구조는 다음 세 가지로 구성하였다.

모델	구조적 특징
SSDLite320 MobileNetV3-Large	Default box를 사용하는 경량 single-stage 구조 (SSD, MobileNetV3)
Faster R-CNN ResNet50-FPN v2	RPN·RoI head와 다중 스케일 feature pyramid를 사용하는 two-stage 구조 (Faster R-CNN, FPN)
YOLO26s	경량 detection head와 다중 스케일 특징을 사용하는 실시간 탐지 구조 (YOLO26, 공식 문서)

SSDLite320과 Faster R-CNN은 각각 ImageNet으로 사전학습한 MobileNetV3-Large와 ResNet-50 backbone을 사용하고, detection head는 background와 pill 클래스에 맞게 새로 학습하였다. YOLO26s는 사전학습된 yolo26s.pt에서 시작해 단일 pill 클래스에 맞게 fine-tuning하였다.

탐지 학습 데이터는 8,143장·bbox 31,238개의 train과 2,026장·bbox 7,771개의 validation으로 구성하였다. SSDLite320과 Faster R-CNN은 별도 증강 없이 학습하였고, YOLO26s에는 방향·색상·위치·크기 변화와 mosaic을 적용하였다. 마지막 5 epoch에는 mosaic을 해제해 실제 이미지 분포에 맞춰 수렴하도록 하였다.

4.3. 분류기 모델 구성 및 학습

분류기는 탐지 bbox에서 잘라낸 알약 이미지를 입력받아 세부 품목을 예측한다. 대회 Train에 포함되지 않은 품목이 Test에 등장할 가능성을 고려해, 본 실험에서는 원천 조합 데이터에 정의된 118개 품목 전체를 분류 대상으로 사용하였다.

초기에는 약 5,000종 분류와 Seen/Unseen gate를 계획했으나, 원천 조합 데이터의 118종으로 범위를 정하면서 118-way 단일 softmax로 단순화하였다.

Backbone은 ImageNet IMAGENET1K_V2로 사전학습한 ResNet-50을 사용하고, 마지막 fully connected layer를 class map의 클래스 수에 맞게 교체하였다. ResNet의 residual connection은 깊은 네트워크의 최적화를 용이하게 하며, 사전학습 특징을 비교적 작은 품목 데이터에 전이하기 적합하다. (Deep Residual Learning for Image Recognition)

예측 bbox는 실제 알약의 경계와 조금 어긋날 수 있어 그대로 자르면 알약 테두리가 잘릴 수 있다. 이를 줄이기 위해 bbox를 각 방향으로 5% 확장한 뒤 crop하였다. 실제 경계와 예측 bbox 사이에 2%의 오차가 있다고 가정한 시뮬레이션에서, 테두리가 잘린 crop의 비율은 여백이 없을 때 93.7%였지만 5% 여백을 적용하면 2.5%로 감소하였다. 반면 주변 알약이

함께 포함되는 비율은 1.6%에서 2.7%로 소폭 증가하는 데 그쳐, 잘림을 방지하면서 주변 영역의 유입을 제한할 수 있는 값으로 5%를 선택하였다.

여백을 포함해 자른 이미지는 원본 비율을 유지한 채 짧은 축에 padding을 추가하여 정사각형으로 만들었다. 이 때 생긴 빈 영역은 RGB (124, 116, 104)로 채웠다. 이 값은 ImageNet 평균색을 8-bit로 환산한 것으로, 정규화 후 padding 영역이 입력에 미치는 영향을 줄인다. 완성된 정사각형 이미지는 224×224로 resize한 뒤 ImageNet mean과 standard deviation으로 정규화하였다. 학습 시에는 알약의 방향 변화에 대응하기 위해 임의 회전만 적용했으며, 각인과 색상 정보를 보존하기 위해 flip과 강한 color jitter는 제외하였다. Validation과 추론에는 무작위 증강 없이 동일한 crop·padding·resize·정규화 과정을 적용하였다.

학습은 다음 두 단계로 진행하였다.

1. 처음 3 epoch는 backbone과 BatchNorm을 동결하고 새 분류 head만 학습하였다.
2. 이후 layer4와 분류 head를 fine-tuning하되 서로 다른 learning rate를 적용하였다.

Loss는 다중 클래스 단일 정답 문제에 맞춰 cross-entropy를 사용하고, 클래스별 표본 수 차이는 WeightedRandomSampler로 보완하였다. 최적 checkpoint는 validation macro F1을 우선하고 동률이면 Top-1 accuracy가 높은 epoch로 선택하였다.

4.4. 탐지·분류 결과 결합 및 후처리

탐지 결과 중 score threshold를 통과한 pill bbox를 분류 입력 후보로 삼는다. 각 bbox에는 분류기 학습과 동일한 여백을 적용한 뒤 이미지 경계 안에서 crop하며, 이 과정에서 좌표가 이미지 경계를 벗어나지 않게 유효

한 범위로 보정한다. 이렇게 생성한 crop을 분류기에 입력한다.

각 bbox에는 분류기가 예측한 품목을 부여하고, 탐지 score와 분류 score를 결합해 최종 신뢰도를 계산한다.

final_score = detector_score × classifier_score

이 방식은 위치 탐지와 품목 분류 중 어느 한 단계의 확신도가 낮으면 최종 score도 함께 낮아지게 한다. 결합된 결과는 bbox, 품목 ID와 score를 포함한 Kaggle 제출 형식의 CSV로 변환한다.

탐지기와 분류기를 연결했을 때의 전체 성능을 확인하기 위해, 학습에 사용하지 않은 validation 조합 이미지 2,026장과 알약 7,771개를 대상으로 end-to-end 검증을 수행하였다. 이를 통해 개별 모델의 성능뿐 아니라 탐지 누락과 위치 오차가 후속 분류에 미치는 영향까지 함께 평가하였다. 예측과 정답은 IoU 0.50을 기준으로 연결해 correct, misclassified, false_positive, false_negative로 구분하고 오류 원인을 분석하였다. 최종 성능은 품목과 bbox 위치를 함께 반영하는 category-aware mAP@[0.75:0.95]로 평가하였다.

4.5. 주요 학습 설정

항목	**SSDLite320**	**Faster R-CNN**	**YOLO26s**	**ResNet-50** 분류기
초기 가중치	ImageNet IMAGENET1K_V1 backbone	ImageNet IMAGENET1K_V2 backbone	yolo26s.pt	ImageNet IMAGENET1K_V2
입력 크기	320×320	짧은 변 384, 긴 변 최대 640	640×640	224×224
Batch size	4	1	4	32
최대 epoch	50	20	30	30, head-only 3 epoch 포함
Optimizer	SGD	SGD	auto가 선택한 AdamW	AdamW
주요 learning rate	0.005	0.0025	0.002	head-only FC 1e-3, fine-tuning layer4 1e-4·FC 5e-4
Weight decay	5e-4	5e-4	5e-4	1e-4
최적 모델 기준	validation mAP@[0.75:0.95]	validation mAP@[0.75:0.95]	validation mAP@[0.50:0.95]	macro F1, 동률이면 Top-1

5. 실험 및 성능 개선

실험은 최초 기준 모델의 오류를 분석한 뒤 데이터 품질과 탐지 모델을 순차적으로 개선하는 방식으로 진행하였다. 성능은 분류기 단독, 탐지기 단독, end-to-end의 세 수준으로 구분하였다.

평가 수준	주요 지표	확인 목적
분류기 단독	Top-1, macro F1	정답 bbox crop의 품목 분류 성능
탐지기 단독	COCO mAP, recall	단일 pill 클래스의 위치 탐지 성능
End-to-end	Category-aware mAP, F1	bbox 위치와 약품 품목을 결합한 최종 성능

탐지 평가는 여러 IoU 기준에서 precision-recall을 집계하는 COCO 방식을 따랐다. (Lin et al., 2014, COCO Detection Evaluation) 모델 비교에는 각 탐지기의 최적 checkpoint와 동일한 ResNet-50 분류기를 연결하였다.

5.1. 기준 모델 및 초기 성능

최초 기준 파이프라인은 경량 탐지기 SSDLite320과 ResNet-50 분류기의 조합으로 구성하였다. Annotation을 정제하기 전 validation에서 측정한 초기 결과는 다음과 같다.

지표	초기 결과
분류기 Top-1 / macro F1	0.9983 / 0.9983
End-to-end F1	0.9942
Category-aware mAP@[0.75:0.95]	0.9088
AP@0.95	0.6294
FP / FN	39 / 46

정상적으로 매칭된 bbox의 분류 오답은 3개뿐이어서 품목 분류보다 bbox 위치가 성능에 더 큰 영향을 주었다. 특히 AP@0.95가 크게 낮아 엄격한 위치 정밀도에 개선 여지가 있었다. 오류 이미지를 확인한 결과 실제 알약은 올바르게 예측했지만 GT bbox가 빈 배경이나 다른 위치에 있어 FP와 FN으로 동시에 집계된 사례도 다수 발견되었다. 따라서 이 결과는 최종 성능이 아니라 데이터 정제와 탐지기 개선 방향을 정한 초기 기준선으로 사용하였다.

5.2. 주요 실험과 변경 사항

초기 분석을 바탕으로 annotation 정제, SSDLite320 재학습, Faster R-CNN과 YOLO26s 비교, score threshold 검토를 진행하였다. 분류기는 정답 crop과 탐지 crop 모두에서 높은 정확도를 보여 동일한 ResNet-50을 유지하였다.

데이터 정제와 **SSDLite320** 재학습

초기 오류 사례를 원본 이미지와 대조하여 다른 알약이나 빈 배경에 놓인 bbox를 학습 대상에서 제외하였다. 정제 후에도 학습 대상인 118개 품목은 모두 유지되었다.

데이터 경로와 분할 정보가 여러 파일과 폴더에 중복되어 같은 데이터를 서로 다르게 해석할 가능성도 있었다. 이에 이미지와 annotation의 기준 위치를 일원화하고 split_manifest.csv를 train·validation 분할의 단일 기준으로 정한 뒤, 정제된 학습 패키지를 다시 생성하였다.

기존 ResNet-50을 정제된 validation에서 다시 평가한 결과 오분류는 발생하지 않았다. 다만 모델을 새로 학습한 결과가 아니라 오류 데이터 정제 내역이 반영된 결과이므로, 이를 분류기의 일반화 성능 향상으로 해석하지 않았다. 이어 정제 데이터로 SSDLite320을 재학습하여 후속 탐지 모델을 평가할 기준선을 마련하였다. 객체 누락은 거의 없었지만 320×320 입력에서는 높은 IoU 구간의 위치 오차가 남았다.

추가 개선의 우선순위를 정하기 위해 별도 SSD 진단 실행에서 예측 품목을 정답으로 대체하였다. 분류 오답을 모두 제거해도 category-aware mAP는 0.9412에서 0.9419로 0.0007만 상승한 반면, 탐지기의 AP@0.95에는 더 큰 개선 여지가 있었다. 이에 제한된 실험 시간은 ResNet-50을 다시 학습하기보다 탐지기의 위치 정밀도를 개선하는 데 우선 배분하였다.

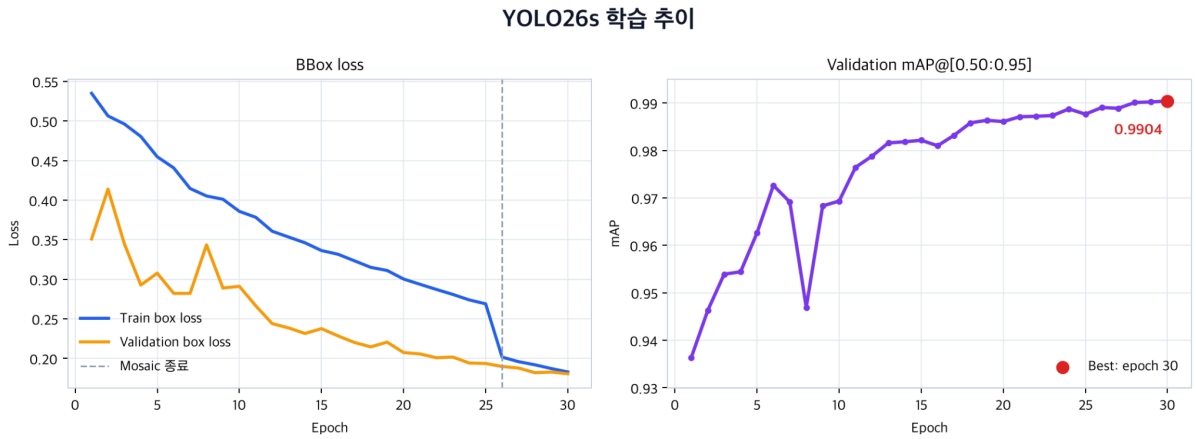
탐지 모델 변경

SSDLite320은 IoU 기준이 엄격해질수록 성능이 하락했으며, 특히 AP@0.95의 낮은 위치 정밀도가 주요 한계였다. 이를 개선하기 위해 더 높은 해상도의 입력과 FPN의 다중 스케일 특징을 활용할 수 있는 Faster R-CNN을 도입하였다. 그러나 높은 해상도에서는 학습 시간이 지나치게 길어 실제 실험에서는 입력을 짧은 변 384px, 긴 변 최대 640px로 조정하고 batch size를 1로 줄여야 했다. 그 결과 SSDLite320보다 높은 IoU 구간의 위치

정밀도는 개선되었지만, 기대했던 고해상도의 이점을 충분히 활용하지 못했고 특정 품목에 FN이 집중되어 학습 시간 대비 결과도 만족스럽지 않았다.

Faster R-CNN에서 확인한 긴 학습 시간과 FN 문제를 보완하면서 높은 IoU의 위치 정밀도를 유지하기 위해 YOLO26s를 추가로 실험하였다. 경량 모델인 YOLO26s는 640×640 입력과 다중 스케일 특징을 사용하면서도 single-stage 방식으로 학습·추론 비용을 줄일 수 있어, 정확도와 효율의 균형을 확인하기에 적합했다. 실제로 validation의 모든 GT를 탐지하면서 Faster R-CNN보다 학습 시간을 줄였고, 높은 IoU 구간의 위치 정밀도도 개선하였다. 남은 탐지 오류는 실제 알약의 누락보다 이미지 경계를 알약으로 판단한 FP에 집중되었다.

그림 5-1. YOLO26s 학습 추이



** Bbox loss*가 전반적으로 감소하고 *validation mAP@0.50:0.95*가 후반까지 상승하였다. *30 epoch*에서 가장 높은 검증 *AP*를 기록한 체크포인트를 선택하였다.

Score threshold 검토

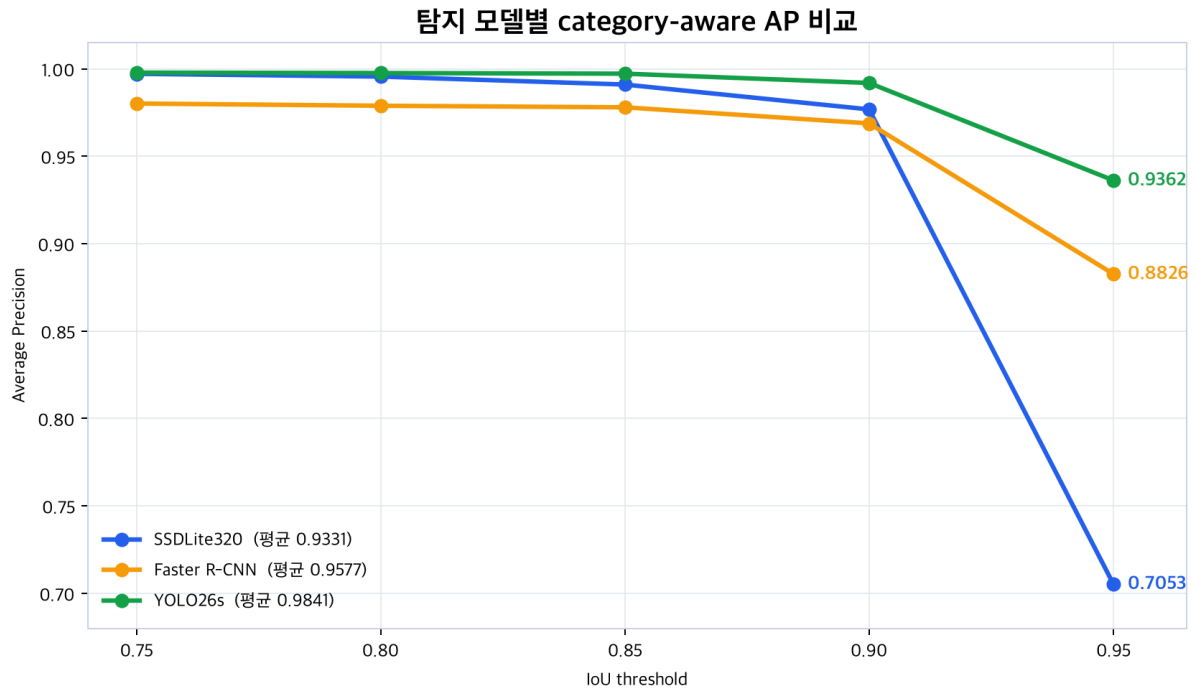
SSDLite320에서는 기준값 0.25보다 조금 낮은 정상 bbox가 제거되어 FN이 발생했다. 반면 YOLO26s는 정상 예측과 경계 FP의 score 구간이 뚜렷하게 분리되었다. Validation에서는 threshold 0.55가 FP 제거 후보였지만, 특정 split에 맞춘 값일 수 있으므로 공통 비교에는 threshold 0.25를 유지하였다.

5.3. 실험별 성능 비교

세 모델의 탐지기 단독 지표는 서로 다른 평가 구현에서 산출되었으므로 직접 비교하지 않았다. 최종 비교에는 동일한 파이프라인으로 계산한 category-aware 결과를 사용하였다.

모델	**Category mAP@[0.75:0.95]**	**AP@0.95**	**End-to-end F1**	오류: 분류 / **FP** / **FN**	학습 시간
SSDLite320	0.9331	0.7053	0.9995	3 / 1 / 1	약 1시간 37분
Faster R-CNN	0.9577	0.8826	0.9935	4 / 7 / 86	약 12시간 15분
YOLO26s	0.9841	0.9362	0.9992	2 / 9 / 0	약 5시간 56분

그림 5-2. 탐지 모델별 category-aware AP 비교



* 동일한 *end-to-end* 평가에서 *IoU* 기준별 *AP*를 비교하였다. *SSDLite320*은 *AP@0.95*에서 크게 하락했고, *Faster R-CNN*은 이를 개선했으며, *YOLO26s*가 높은 *IoU* 구간에서 가장 높은 *AP*를 유지하였다.

SSDLite320의 F1이 가장 높은 반면 YOLO26s의 mAP가 가장 높은 것은 두 지표가 강조하는 요소가 다르기 때문이다. F1은 IoU 0.50에서 집계한 오류 개수에 민감하고, mAP는 여러 IoU 기준에서 bbox 위치와 예측 순위를 함께 반영한다.

Kaggle 제출 실험

모델 비교와 별개로 채점 방식과 후처리의 영향을 확인하는 제출 실험을 수행하였다. 각 제출은 기준에서 한 가지 가설을 확인하도록 구성하고 테스트 842장 전체에 예측을 생성하였다.

변경	기준 → 결과	관찰
데이터 범위 확대	대회만 0.37884 → 탐지기 v1 0.58186	추가 데이터 확보 후 상승
정제 반영	v1 0.58186 → v2 진단 기준 0.58765	검수 반영 후 상승
탐지기 교체	SSD v2 0.58765 → YOLO26s 0.61804	더 높은 Public 점수
박스 일괄 ±3%	0.49796 / 0.50087	박스 크기 변화에 민감

증강 A1	0.58765 → 0.58430	로컬 상승이 외부 상승으로 이어지지 않음
후처리·앙상블	대체로 ±0.003 이내	점수 변화가 작음

세부 제출·가설·점수는 부록 A에 모았다. SSD v2 진단 기준 0.58765와 팀원 단독 제출 0.58964는 별도 제출로 구분하였다. YOLO11s 단독 0.60831, Faster R-CNN 0.59375, YOLO26s 단독 0.61804도 제출 기록으로 보존하였다. 로컬 공통 비교표와 제출 실험은 평가 데이터·목적이 달라 직접 섞지 않았다.

5.4. 최종 모델 선정 근거

대회 평가 지표가 mAP@[0.75:0.95]이므로 높은 IoU 구간의 category-aware 성능을 우선하였다. YOLO26s는 세 모델 중 category-aware mAP와 AP@0.95가 가장 높고 FN이 없었으며, Faster R-CNN보다 학습 시간도 짧아 최종 탐지기로 선정하였다.

분류기는 YOLO26s가 탐지한 bbox에서 0.9997의 정확도를 기록했고 오답도 외형이 유사한 캡슐 두 건에 한정되었다. 따라서 ResNet-50을 유지하고 YOLO26s와 결합한 구성을 최종 파이프라인으로 선정하였다.

6. 최종 결과 및 분석

6.1. 최종 모델의 검증 성능

최종 파이프라인은 YOLO26s 탐지기와 ResNet-50 분류기의 조합이다. 정제된 validation 2,026장에 포함된 GT 알약 7,771개를 대상으로 평가하였다.

평가 구분	지표	결과
탐지기 단독	mAP@[0.50:0.95]	0.9904
탐지기 단독	Precision / Recall	0.99999 / 1.0000
End-to-end	Category-aware mAP@[0.75:0.95]	0.9841
End-to-end	AP@0.95	0.9362
End-to-end	Precision / Recall / F1	0.9986 / 0.9997 / 0.9992

평가 구분	지표	결과
End-to-end	탐지 bbox 분류 정확도	0.9997

단일 pill 클래스로 평가한 탐지기는 mAP@[0.50:0.95] 0.9904와 recall 1.0000을 기록해 validation의 알약을 안정적으로 찾았다. 품목 분류까지 결합한 end-to-end 평가에서도 precision 0.9986, recall 0.9997, F1 0.9992와 분류 정확도 0.9997을 기록하여 객체 누락과 품목 오답은 매우 적었다.

다만 품목과 bbox 위치를 함께 평가한 AP@0.95는 0.9362로 다른 지표보다 낮아, 매우 엄격한 IoU 조건에서는 위치·크기 오차가 일부 남아 있음을 확인하였다. 탐지기 단독 지표와 end-to-end 지표는 평가 대상이 다르므로, 최종 모델 판단에는 대회 평가 방식과 같은 category-aware mAP@[0.75:0.95] 0.9841을 우선하였다.

6.2. Kaggle 제출 결과

최종 파이프라인을 test 이미지 842장에 적용해 Kaggle 제출 형식의 submission.csv를 생성하였다.

항목	결과
Test 이미지	842장
제출 예측	3,229개
이미지당 예측	3~4개, 평균 3.83개
Detector threshold	0.25

제출 파일은 대회가 요구한 열과 형식을 갖추었으며 모든 test 이미지에 예측이 포함된 것을 확인하였다.

최종 제출의 Kaggle Public mAP@[0.75:0.95]는 0.61804로, 로컬 검증 결과보다 크게 낮았다. 운영진 답변을 통해 Public 정답에 포함된 74개 품목의 AP를 평균한다는 채점 방식을 확인했으며, 정답에 없는 품목까지 분모에 포함돼 점수 상한이 낮아진다는 초기 가설은 기각하였다.

품목별 결과와 후처리 변경을 비교한 결과, bbox 복제나 score 순위 조정보다 탐지기를 바꿔 bbox 위치가 달라졌을 때 Public 점수의 변화가 더 크게 나타났다. 육안상 예측이 적절해 보이는 품목에서도 AP가 낮게 나타나 높은 IoU 구간의 위치 차이를 우선 점검하였다.

별도 SSD 증강 실험에서도 로컬 validation은 0.0096 상승했지만 Public 점수는 0.58765에서 0.58430으로 하락했고, 대회 출처 이미지의 AP@0.95가 AI Hub 출처보다 낮았다. 이는 현재 validation이 Public 분포를 충분히 대표하지 못했을 가능성을 보여준다.

종합하면 로컬·Public 성능 차이는 높은 IoU 구간의 위치 오차와 두 데이터의 분포 차이가 함께 영향을 주었을 가능성이 크다. 누락 annotation이나 Public/Private 분할 방식의 영향도 검토했지만, 공개 정답이 없어 원인으로 확정하지 않았다.

6.3. 예측 결과 시각화

최종 파이프라인이 한 이미지의 모든 알약을 탐지하고 각 bbox에 세부 품목을 부여하는 과정을 확인하기 위해 validation의 정상 예측 사례를 시각화하였다.

그림 6-1. YOLO26s와 ResNet-50의 정상 예측 사례



* 초록 점선은 *GT bbox*, 파란 실선은 *bbox*와 품목이 모두 일치한 최종 예측이다. 네 알약을 모두 올바르게 탐지·분류했으며 예측 *bbox*도 *GT*와 가깝게 겹친다.

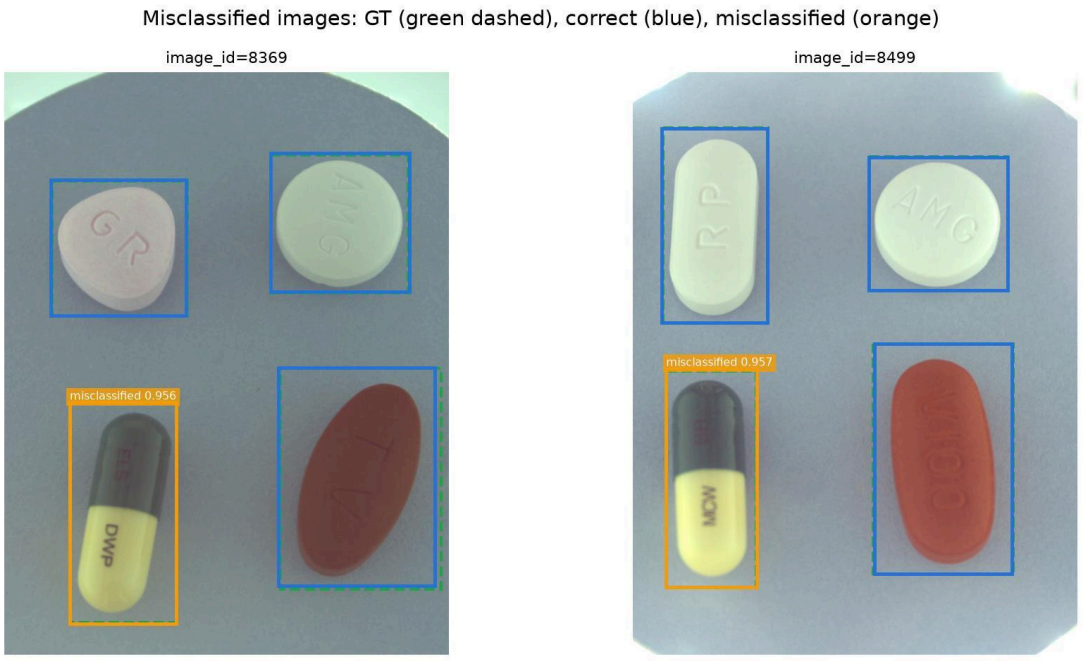
이 그림은 탐지 결과가 분류기로 전달되어 최종 품목 예측으로 결합되는 형태를 정성적으로 보여준다. 단일 사례는 성능의 근거로 사용하지 않았으며, 최종 판단에는 전체 validation 지표와 오류 사례 분석을 함께 사용하였다.

6.4. 오류 사례 및 원인 분석

최종 validation 오류는 분류 오답 2개와 false positive 9개였으며 false negative는 없었다.

오류 유형	건수	주요 원인
분류 오답	2	외형이 유사한 에르도스테인 캡슐 간 혼동
False positive	9	이미지 위·아래의 얇은 경계를 알약으로 탐지
False negative	0	validation의 모든 GT가 예측과 매칭됨

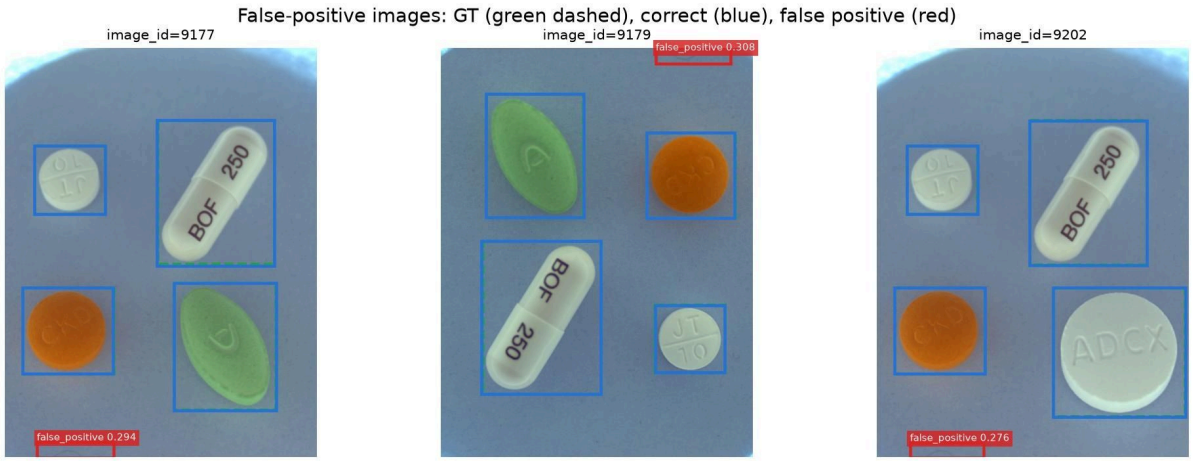
그림 6-2. YOLO26s end-to-end 분류 오답



* 주황색은 품목 분류가 틀린 ***bbox***다. 엘스테인캡슐과 뮤코원캡슐을 서로 반대로 예측한 두 사례로, ***bbox IoU***는 모두 ***0.97*** 이상이었다.

두 오답의 classifier score는 각각 0.5038과 0.5089로 결정 경계에 가까웠다. 두 bbox의 IoU가 모두 높았으므로 탐지 위치보다 유사 품목의 색상·각인 차이를 구분하는 문제가 원인으로 판단되었다.

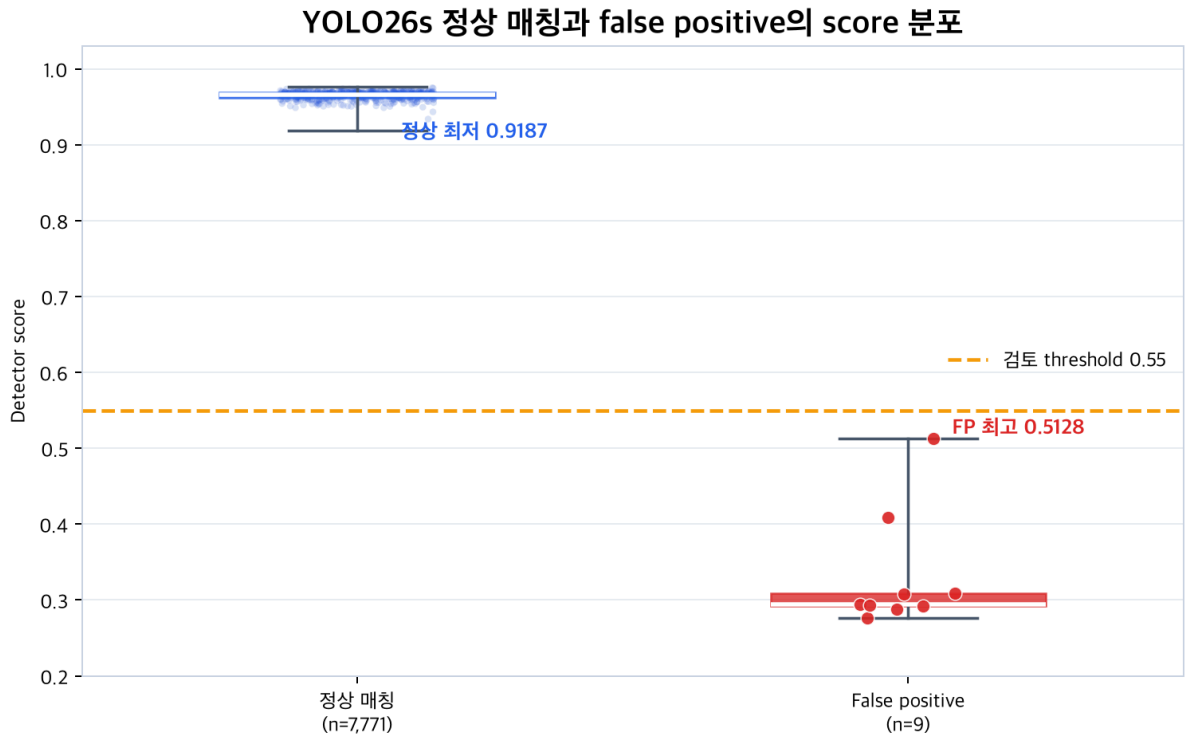
그림 6-3. YOLO26s 경계 false positive 대표 사례



* 전체 *FP 9*건 중 대표적인 *3*건이다. 빨간색 *FP*가 실제 알약 주변이 아니라 이미지 위·아래 경계에 나타나는 공통 패턴을 보였다.

FP 9개의 detector score는 0.2759~0.5128인 반면 정상 매칭의 최저 score는 0.9187이었다.

그림 6-4. YOLO26s 정상 매칭과 false positive의 score 분포



* *Validation*에서는 정상 매칭과 *FP*의 *detector score*가 분리되었으며, 검토한 *threshold 0.55*는 두 구간 사이에 위치하였다.

현재 validation에서는 threshold를 0.55로 높이면 정상 매칭을 유지하면서 FP를 제거할 수 있었다. 다만 하나의 split에서 확인한 결과만으로 threshold를 변경하기에는 근거가 부족해, 공통 모델 비교에는 기존 값인 0.25를 유지하였다.

분류 혼동은 유사 품목의 hard example을 보강해 개선해볼 수 있다. 경계 FP는 score threshold를 다시 조정하거나 위치·종횡비를 함께 고려하는 후처리를 실험할 수 있지만, 실제 경계 알약을 제거하지 않는지 추가 검증이 필요하다. 두 방법 모두 오류 분석에서 도출한 개선 후보이며 최종 모델에는 적용하지 않았다.

SSD 별도 진단과 개선 우선순위

위의 최종 YOLO26s 오류와 별개로 SSD v2 진단 실행에서는 매칭 박스 7,770개 중 분류 오답 6건(정확도 0.9992)이 확인되었다. 엘스테인·뮤코원·알바스테인 캡슐 사이의 혼동이며 분류 점수는 0.65~0.75였다. SSD A1에서는 분류기를 바꾸지 않아도 6건이 모두 정답으로 바뀌어, 해당 실행에서 분류기를 추가 학습하기보다 탐지 위치 정밀도를 우선하기로 하였다.

SSD의 작은 알약 누락은 Faster R-CNN·YOLO에서 같은 방식으로 나타나지 않아 SSD 320×320 입력의 한계로 해석하고, SSD만을 위한 추가 데이터셋 개선은 진행하지 않았다. 이 해석은 모델 간 관찰에 따른 것이며 모든 작은 알약 누락의 원인을 일반화한 것은 아니다.

7. 협업 및 프로젝트 운영

7.1. 협업 흐름

정기 회의는 Notion에서 진행하고 진행 상황, 논의 사항과 결정 내용을 기록하였다.
회의에서 필요한 작업을 정하면 목적·담당자·완료 조건이 포함된 할 일 카드를 만들고, 실제 코드 작업으로 이어지는 카드는 같은 내용을 바탕으로 GitHub Issue를 생성하였다.

Notion 회의·기록 → 할 일 카드 작성 → GitHub Issue 생성 → 작업 브랜치 구현·검증 → PR 리뷰·병합 → Issue 종료 및 Notion 결과 갱신

이 흐름을 통해 회의에서 결정한 내용이 구두 합의로 끝나지 않고 담당 작업과 코드 변경으로 연결되도록 하였다. 작업 완료 후에는 Issue와 Notion 카드를 함께 정리하여 진행 상태와 결과를 일치시켰다.

약 2주 동안 3명이 작업하므로 중간 통합 브랜치를 별도로 두는 Git Flow보다 단순한 GitHub Flow가 병합 관리에 적합하다고 판단하였다. PR 리뷰는 승인 절차뿐 아니라 팀원 모두가 main에 반영된 코드의 동작을 이해하는 것을 목표로 하였다.

7.2. Git 규칙 및 작업 양식

팀원이 같은 방식으로 작업을 시작하고 검토할 수 있도록 GitHub 협업 가이드와 작성 양식을 합의하였다.

관리 항목	약속한 규칙과 양식
작업 시작	최신 main을 동기화한 뒤 새 작업을 시작하고, main에는 직접 commit하거나 push하지 않음
Issue	목적·작업 내용·완료 조건을 작성하고 담당자를 지정
브랜치	최신 main에서 이슈번호-작업명 형식으로 생성하고 작업명은 소문자 영문과 하이픈 사용
Commit	feat:, fix:, chore:로 변경 목적을 구분하고 한글로 간결하게 작성
PR	주요 작업 내용·검증 결과·관련 Issue를 기록하고 main을 대상으로 생성
Review	최소 1명의 승인을 받고, PR 제출자가 최신 main과의 호환성을 먼저 확인
Merge	작업 특성에 따라 Squash Merge 또는 개별 commit을 보존하는 일반 merge 선택
작업 종료	병합 후 Issue를 종료하고 사용한 원격·로컬 작업 브랜치를 정리

초기에는 commit 이력을 간결하게 유지하기 위해 Squash Merge를 사용하였다. 이후 큰 리팩터링과 데이터 통합을 여러 단계로 검증하면서 작업 과정 자체를 기록할 필요가 커져, 개별 commit을 보존하는 일반 merge 방식으로 전환하였다.

7.3. 핵심 코드 구조 및 실험 관리

코드는 원천 데이터 처리, 학습 입력 구성, 모델 실행과 통합 추론의 책임이 섞이지 않도록 구분하였다.

경로	핵심 역할
etl/	원천 데이터 검수·정제·분할과 학습 패키지 생성을 담당하는 오프라인 처리
data_loading/	준비된 학습 패키지를 읽어 Dataset, DataLoader와 tensor로 변환
classifier/	품목 분류 모델의 생성, 학습, 평가, 추론과 checkpoint 관리

경로	핵심 역할
detector/	단일 pill 클래스 탐지 모델의 생성, 학습, 평가, 추론과 checkpoint 관리
pipeline/	탐지 bbox를 분류 입력으로 변환하고 두 모델의 결과를 결합해 end-to-end 평가·추론 수행
configs/	Notebook과 CLI가 공유하는 실행 설정, YAML 로딩과 값 검증
main.py·notebooks/	재현 가능한 CLI 실행과 baseline·experiment·EDA 목적의 대화형 실행 진입점
utils/·tests/	장치·seed·파일 저장 등 공통 기능과 단위·smoke·통합 검증

원천 데이터 → etl/ 검수·정제·학습 패키지 생성 → data/ 학습 패키지 → data_loading/ Dataset·DataLoader 변환 → classifier/·detector/ 학습·평가·추론 → pipeline/ 탐지와 분류 결과 결합

분류기와 탐지기는 모델 생성·학습·평가·추론·checkpoint 관리처럼 공통된 책임을 유사한 구조로 배치하였다. 반면 분류기의 단계별 fine-tuning과 탐지기의 bbox 평가처럼 작업 특성이 다른 로직은 억지로 공통화하지 않고 각 모듈에 유지하였다. Notebook과 main.py는 학습 로직을 다시 구현하지 않고 이 Python 모듈을 호출하도록 구성하였다.

실행 환경은 Python 3.12와 uv로 관리하였다. .python-version, pyproject.toml, uv.lock을 공유하고 uv sync --locked로 같은 의존성 환경을 구성했으며, 패키지 변경 시 uv add 또는 uv remove를 사용해 설정 파일과 lock file을 함께 PR에 포함하였다. Notebook은 nbstripout으로 출력을 제거하고 필요할 때 nbdime으로 변경 사항과 충돌을 확인하였다.

실험은 날짜·모델·데이터·목적을 포함한 run_name으로 구분하고 outputs/<task>/<run_name>에 checkpoint, 평가 지표와 실제 적용 설정을 저장하였다. 공통 seed는 42로 설정하고 resolved_config.json을 남겼으며, 핵심 비교표와 그래프·해석은 reports/에 기록하였다. 전체 학습 전에는 합성 데이터와 smoke test로 실행 경로를 먼저 확인하였다.

실험 tag는 이후 W&B 연동 시에도 사용할 수 있도록 key:value 형식으로 정하였다. W&B 연결 계획과 실제 적용 여부는 구분하여 기록하였다.

7.4. 커뮤니케이션을 통한 문제 해결

작업 중 문제가 발생하면 담당자가 개별적으로 처리하기보다 Notion 회의에서 현상과 영향 범위를 먼저 공유하였다. 원인을 데이터·모델·코드로 구분하고 로그, 지표와 시각화 자료를 함께 확인해 대응 방법과 우선순위를 결정하였다. 공통 데이터 형식이나 모델 인터페이스처럼 다른 작업에 영향을 주는 변경은 구현 전에 담당자와 완료 조건을 합의하였다.

Merge conflict가 발생했을 때는 한쪽 변경을 바로 덮어쓰지 않고 각 변경의 의도를 확인한 뒤 통합하였다. 모델 선택과 데이터 정제처럼 판단이 필요한 경우에는 수치와 오류 사례를 근거로 합의하고, 결정 내용과 검증 결과를 Notion·Issue·PR·보고서에 남겨 이후 작업에서도 같은 기준을 사용할 수 있도록 하였다.

희소 품목의 추가 데이터가 대용량 zip으로만 제공되는 제약은 데이터 확보 범위와 학습 방식을 함께 조율한 사례였다. 413GB 추가 다운로드 대신 기존 표본의 역수 가중 샘플링을 활용하였으며, 이는 표본 부족을 완전히 해결한 것은 아니었다(3.1).

7.5. 일정 및 역할 조정

역할은 모델과 데이터의 주 담당을 구분하되, 학습 단계에서는 탐지 모델을 나누어 병렬로 진행하였다.

담당	주요 역할
대현	탐지기·분류기 모델 구현과 두 모델을 연결하는 end-to-end 파이프라인 작업
의연	데이터 정제와 annotation 검수, 데이터 분포 및 오류 사례 분석
홍식	팀장으로서 진행 상황 조율과 보조 작업을 담당하고, 최종 보고서 정리와 발표 준비 수행
공동	탐지기 후보 모델을 나누어 학습하고 결과와 오류 사례를 공유해 최종 모델 비교

전체 일정은 준비·기획 → 데이터 분석 → 1차 모델 구현 → 보완·고도화 → 결과물 정리의 다섯 마일스톤으로 구성하였다.

주차	마일스톤	주요 작업
1주차	준비·기획	문제 정의, 2단계 예측 구조와 협업 방식 확정, 입력 데이터 명세 작성
2주차	데이터 분석	annotation과 데이터 분포를 검토하고 오류 데이터를 정제
2주차	1차 모델 구현	기준 탐지기와 분류기를 구현하고 학습·평가·추론 흐름 연결
3주차	보완·고도화	탐지 모델을 추가 학습하고 end-to-end 성능과 오류 유형 비교
3주차	결과물 정리	최종 추론과 제출 결과를 점검하고 보고서와 발표 자료 정리

탐지기 데이터 로더 통합이 예상보다 늦어지면서 전체 학습과 비교 실험의 시작도 다소 지연되었다. 남은 기간에는 모델별 반복 학습과 추가 설정 탐색의 규모를 줄이고, 주요 탐지

모델 비교와 end-to-end 검증, 최종 결과물 완성에 우선순위를 두었다. 이를 통해 제한된 일정 안에서 실험 범위를 조정하면서도 최종 파이프라인과 분석 결과를 마무리하였다.

8. 결론 및 회고

8.1. 프로젝트 성과 및 목표 달성도

추가 데이터 확보와 검수를 통해 탐지기 v2 10,169장·39,009박스 및 분류기 v2 14,798 crop을 구성하였다. 위치 탐지와 품목 분류를 분리한 파이프라인으로 여러 탐지기를 공통 분류기와 비교하고, 최종 YOLO26s + ResNet-50 구성을 정리하였다.

최종 통합 검증 mAP@[0.75:0.95]는 0.9841, 자료상 Public 점수는 0.61804였다. 내부 성능과 외부 점수의 차이가 남아 목표 달성도를 검증 점수만으로 평가하지 않았다. 데이터 오류를 추적하고 채점 가설을 검증하는 과정도 주요 수행 결과로 정리하였다.

SSDLite320·Faster R-CNN·YOLO11s·YOLO26s를 후보로 검토하고, 동일한 분류기를 연결하는 구조로 탐지기 교체와 오류 분석을 수행하였다. 검수 고유 오류 59개를 확인한 분류기 검수와 탐지기 추가 검수는 범위를 구분하여 기록하였다.

8.2. 한계 및 개선 방향

내부 검증은 대부분 AI-Hub 출처여서 Kaggle 테스트를 충분히 대표하지 못할 수 있었다. 추가 증강·후처리의 로컬 개선이 Public 개선으로 이어지지 않은 점을 근거로 다음 과제를 정리하였다.

한계·경험	개선 방향
AI-Hub 중심 검증 분포	대회 출처 이미지만의 별도 평가와 테스트 유사 검증 조건 구성
높은 IoU의 위치 오차	입력 해상도, IoU 계열 박스 손실, 보정 단계를 동일 조건에서 비교
학습부터 제출까지 연결 지연	소형 데이터로 학습→추론→제출을 초기에 연결한 뒤 확장
희소 품목 부족	부분 확보 방법과 비용을 초기에 확인하고 보강·샘플링 효과 검증
A1 기준선과 학습 길이 차이	동일 epoch·종료 조건으로 증강 효과 재비교
유사 품목의 드문 혼동	어려운 사례와 해당 품목 표본 보강
실행별 결과 연결 부족	데이터 버전·체크포인트·후처리·제출 CSV의 대응표 관리
검증 결과를 이용한 반복 개선	별도 보류 세트·독립 평가로 선택 편향 점검

Public 점수 차이를 정답 문제로 단정하지 않고, 데이터 분포·위치 정밀도·평가 구현과 실행 조건을 먼저 점검해야 한다. 누락 라벨이나 Public/Private 구성 방식은 공개 정답이 없어 확인되지 않은 가설로 남겼다.

8.3. 팀원별 회고

[김대현]

분류기와 여러 탐지 모델을 구현하고 end-to-end 파이프라인을 통합하는 과정에서, 모델링뿐 아니라 Git 기반 협업 구조와 역할 분담의 중요성을 크게 배웠다. 다시 프로젝트를 진행한다면 베이스라인을 MVP 형태로 더 빠르게 완성해 대조군으로 삼고, 남은 시간과 연산 자원을 다양한 실험과 성능 개선에 배분하고 싶다.

또한 초기에 팀원별 경험과 강점을 충분히 파악해 역할과 작업 범위를 조정하는 과정이 필요하다고 느꼈다. 진행이 어려운 작업은 결과를 기다리기보다 상황을 일찍 공유하고, 함께 해결하거나 필요한 부분을 적극적으로 지원했다면 전체 일정이 더욱 원활했을 것이다. 다음 프로젝트에서는 작업 진행 상황을 자주 확인하고, 팀원 간 지원이 필요한 시점을 빠르게 판단하는 협업 방식을 적용하고자 한다.

[김의연]

계획한 일정 안에서 각자 맡은 업무를 수행하고 공유하며 함께 의사결정하는 과정과 Git 기반 협업을 경험하면서 개인적·협업적 측면에서 앞으로 더 발전시켜 나갈 부분을 생각해 볼 수 있었다.개인적 측면에서는 낯선 협업 방식과 빠른 진행 속도, AI를 활용한 작업 과정에서 업무를 주도하기보다 흐름에 끌려간다는 느낌을 자주 받았다. 이를 통해 조급할수록 '무엇을, 왜 하고 있으며, 핵심은 무엇인가'를 점검하고, 의사결정의 근거를 스스로 명확히 이해하고 설명할 수 있어야 한다는 점을 깨달았다. 협업적 측면에서는, 프로젝트 전체의 진행 상황을 파악하고 우선순위가 높은 업무를 함께 해결하는 유연성이 필요하다고 느꼈다. 다음 프로젝트에서는 업무의 목적과 핵심을 명확히 이해하며 주도적으로 의사결정하고, 팀 전체의 상황과 우선순위를 고려해 더욱 유연하게 협업하는 데 중점을 둘 것이다.

[진홍식]

경험이나 능력이 서로 다른 사람들끼리 자연스럽게 모여 협업을 할 수 있는 기회가 있어서 좋았고, 그 기회속에서 협업의 전체적인 과정이나 관련 툴을 익히고 업무 수행에 필요한 여러 지식들을 단기간에 많이 배울 수 있었다.

다만 작업을 할 때 부족한 시간과 지식을 메꾸기 위해 사용했던 AI에 과의존하여 완성된 결과물을 팀원들에게 제대로 전달할 수 없었던 점이 매우 아쉬웠다. 아무래도 계속 공부해 나가는 것이 현재는 첫번째 과제인 인 것 같고 다음 프로젝트에서는 적어도 팀원들에게 무엇을 어떻게 전달하지 목적을 가지고 제대로 전달할 수 있는 팀원이 되는 것이 목표이다.

참고 자료 및 부록

참고 자료 및 출처

• Kaggle「[AI15] 경구약제 이미지 객체 검출 프로젝트」: https://www.kaggle.com/competitions/ai15-level1-project

• GitHub Repository: https://github.com/auranova1125/gitkk-pill-detection

• AI-Hub 데이터 분석 및 선정 현황; 탐지기 데이터셋 v2 구성 결과; 2. 데이터 분석(2.1~2.2); 4팀 중점사항, 주요 의사결정; 4, 5, 6, 7장 — 팀 내부 기록

• 모델·평가 참고 자료: Faster R-CNN, Feature Pyramid Networks, Deep Residual Learning for Image Recognition, SSD, MobileNetV3, Double-Head R-CNN, COCO Detection Evaluation, Ultralytics YOLO26 문서

[(논문별 저자·연도·공식 원문 링크, AI-Hub 원천 데이터 링크, 운영진 답변 링크) 보완 예정]

• 추가 통합 근거: 김의연「2, 3장 + 다른 장 추가 사항」(2026. 11. 05.)

부록 **A.** 주요 제출 진단 기록

기준 제출과 진단 제출을 구분하여 기록하였다. 같은 SSD v2 이름이라도 팀원 단독 제출과 진단 기준 제출의 체크포인트·설정이 동일하다고 단정하지 않았다.

실험	변경·가설	**Public**	판단 범위
대회 데이터만	215장 기반 초기 제출	0.37884	학습~제출 연결 기준점
탐지기 v1	AI-Hub 추가	0.58186	이후 진단 기준(sub02)
탐지기 v2	2차 검수 반영	0.58765	진단 제출 기준
박스 +3%	모든 박스 확대	0.49796	일괄 크기 변경에 민감
박스 −3%	모든 박스 축소	0.50087	위와 같음
AI-Hub 전용 18종 제거	대회 학습에 없던 품목의 채점 여부	0.43790	제거가 개선으로 이어지지 않음
기넥신에프정만	369박스 유지	0.00704	74배 환산 AP 약 0.521
하이트린정 2mg만	139박스 유지	0.00688	74배 환산 AP 약 0.509
뮤테란캡슐만	137박스 유지	0.00662	74배 환산 AP 약 0.490
박스 복제	낮은 점수(×0.001)로 복제 추가	0.58280	대량 중복 정답 가설을 지지하지 않음
4알약 이미지 순위 낮춤	701장 박스 점수 ×0.001	0.57731	큰 개선 없음
SSD v2 변형 박스	±1% 예비 박스, 점수 ×0.001	0.59026	로컬 0.9228→0.9297; 외부 변화 작음
YOLO11s 단독	중간 모델 제출	0.60831	단독 기준

YOLO11s 예비 박스	평균·변형 박스 추가	0.60852	단독 대비 약 +0.0002; 통합 val 예측 미확보
YOLO26s 단독	최종 선택 모델	0.61804	자료상 최고 단독 점수
YOLO26s·Faster R-CNN 평균	가중치 0.6 / 0.4	0.61780	단독보다 낮아 미채택
SSD v2 A1	뒤집기·밝기·대비	0.58430	로컬 0.9402→0.9498; 미채택
팀원 SSD v2 단독	진단 기준과 별도 제출	0.58964	실행 연결 확인 필요
팀원 Faster R-CNN	공통 분류기 제출	0.59375	제출 기록

Public은 정답 74종의 AP 평균이라는 운영진 답변을 기준으로 단일 클래스 제출 점수에 74를 곱해 AP를 추정하였다. 118종이 모두 분모에 포함되어 상한이 낮아진다는 가설은 기각하였다. 일부 품목의 AP가 약 0.5였지만, 이것만으로 정답 누락이나 분할 방식을 확정할 수는 없다.

[확인 M05] 0.9228·0.9402·0.9412 등의 SSD 로컬 값은 서로 다른 진단·증강·평가 실행으로 기록되어 있다. 최종 보고서 확정 전에 run_name과 데이터·체크포인트를 연결한다.

부록 **B.** 재현 및 상세 실험 자료

-

부록 **C.** 개인별 협업일지

[(김대현 협업일지 링크) 보완 예정]

[(김의연 협업일지 링크) 보완 예정]

[(진홍식 협업일지 링크) 보완 예정]

부록 **D.** 데이터 검수 세부 기준

1차 검수 **18**개 항목

분류	항목	판정 기준
Annotation	면적 불일치(오류)	area ≠ 가로×세로
Annotation	범위 초과(오류)	이미지 안에 남은 비율 70% 미만 제외, 이상이면 경계에 맞춤
Annotation	음수·0 크기(오류)	가로 또는 세로 ≤ 0
Annotation	ID 중복(오류)	같은 ID 2개 이상
Annotation	동일 bbox 반복(의심)	다른 약 라벨에 같은 좌표
Annotation	과도한 중첩(의심)	IoU > 0.7 또는 작은 박스 90% 이상 겹침
Annotation	ignore·iscrowd(의심)	0이 아닌 값

Image	이미지 ID 중복(오류)	같은 ID 2개 이상
Image	PNG 누락·손상(오류)	파일 부재 또는 열기 실패
Image	크기 불일치(오류)	실제 크기와 JSON width·height 차이
Image	중복·유사(의심)	픽셀 해시 일치, pHash 거리 ≤ 5
교차 검증	코드 매핑(의심)	폴더 코드·drug_N·dl_mapping_code 불일치
교차 검증	파일명·라벨 구성(의심)	약 코드 집합·개수와 라벨 집합·박스 수 불일치
교차 검증	제품 정보 상충(의심)	같은 약 코드에 다른 이름·모양
교차 검증	각도별 객체 수(의심)	같은 조합 70·75·90°의 박스 수 차이
교차 검증	실물·박스 크기(의심)	실물 1mm당 긴 변 18~33px·짧은 변 18px 이상; 실물 길이 없으면 동일 클래스 중앙값의 긴 변 0.7배·짧은 변 0.5배 이상
EDA	크기·종횡비(편향)	클래스 분포의 이상치
EDA	위치 편향(편향)	특정 위치에 집중

범위 초과를 경계에 맞추는 1차 처리는 바운딩 박스를 새로 그리는 재라벨링과 구분하였다. 상세 수치와 적용 단계는 원본 검수 기록으로 확인한다.

주요 검출 결과

분류기 후보 기준 AI-Hub 10,523장 + Kaggle 232장·41,164박스에 대한 기록이다. 모집단 범위는 M01 확인 대상이며, 항목 간 중복이 있으므로 검출 건수를 합산해 고유 제외 수로 사용하지 않았다.

항목	검출	관련 이미지	조치
음수·0 크기	147	122	모두 제외
동일 bbox 반복	142	71	141건 제외
파일명·라벨 구성 불일치	115	66	107건 제외
과도한 중첩	68	34	모두 제외
범위 초과	59	59	48건 제외, 11건 경계 맞춤
PNG 누락·손상	27	27	모두 제외
실물·박스 크기 불일치	1,593	1,569	원본 확인 후 116건 제외, 나머지 기록
면적 불일치	420	405	crop 영향이 없어 기록
중복·유사 이미지	10,704	3,858	각도별 유사 사례가 많아 기록하고 그룹 분할에서 처리

ID·PNG 크기·코드·제품 정보	0	0	해당 이상 없음

2차 검사 알고리즘과 후보 판정

위치 검사는 이미지를 1/4로 줄여 Lab 배경의 조명 기울기를 추정하고 알약을 제외하며 4회 반복 적합하였다. 채도 차×1.5·밝아짐·어두워짐의 최댓값이 12를 넘는 픽셀과 Canny 윤곽선을 합친 뒤 열기 3×3·닫기 7×7을 적용하였다. 이미지 면적 0.4% 미만(가장자리 2% 미만)의 영역은 버리고 헝가리안 알고리즘으로 영역·박스를 1:1 대응하였다.

지표	의미
match_iou	대응 영역과 박스의 IoU; 0이면 대응 없음
fg_in	박스 내부의 알약 영역 비율
ring_fg	박스 바깥 폭 8% 띠의 알약 영역 비율
unmatched_blobs	어느 박스에도 대응하지 않는 영역 수

위치 1순위는 fg_in < 0.15, 또는 0 < match_iou < 0.5, 또는 대응 없음과 함께 남는 영역이 있거나 fg_in < 0.5인 경우였다. 2순위는 대응 없음, 3순위는 match_iou < 0.6이었다. 등급은 자동 후보 우선순위이며 오류 확정값이 아니었다.

품목 검사는 기준 ResNet-50의 crop 임베딩으로 같은 조합의 다른 각도 사진을 비교하고, 자기 조합을 제외한 5-fold 로지스틱 회귀 예측을 함께 사용하였다. 같은 사진의 두 crop이 서로 상대 품목으로 판단되면 swap 후보로 표시하였다. swap 또는 두 검사 공통 표시는 1순위, 한 검사만 표시는 2순위로 분류하였다.

검사·등급	후보	오류	정상
위치 1순위	59	42	17
위치 2순위	31	0	31
위치 3순위	45	10	35
품목 1순위	16	16	0
품목 2순위	19	10	9

위치 오류는 52개·48이미지(train 36 / val 16)로 기록되어 있다. 품목 오류 26개 중 19개는 위치 검사와 공통이며 7개는 품목 검사에서만 확인되어 고유 오류는 59개였다. 붙은 알약을 하나로 찾거나 배경과 비슷한 색의 알약을 놓치는 자동 오탐이 있어 원본 확인을 최종 기준으로 사용하였다.

부록 **E.** 통합 후 확인할 사항

번호	확인 대상	관련 절
M01	10,509 / 10,503 / 10,523장 모집단·시점 차이	2.1·3.1·부록 D
M02	추가 제외 5장과 관찰 오류 14장·제외 17장의 관계	3.1

M03	분류기 라벨 교환 2 crop 및 탐지기 메타데이터 수정 여부	3.1·5.2·부록 D
M05	SSD 실행별 0.9228 / 0.9402 / 0.9412의 대응	5.2·부록 A

[(팀원 확인 결과를 반영한 뒤 확인 표시 제거 및 최종 실행·출처 링크 확정) 보완 예정]

부록 **F.** 운영진 문의 및 답변 내용

AI_15기_김의연 26. 11. 3. 오후 7:13
안녕하세요, 4팀(깃크크)입니다.
초급 프로젝트 Public 점수를 정확히 이해하고 발표에 설명하기 위해 평가 방식에 대해 문의드립니다.

저희가 궁금한 점은 test 정답에 없는 약 종류도 AP를 0점으로 넣어 평균을 내는지입니다.

예를 들어 전체 평가 대상이 118종이고 Public 정답에는 74종만 있다면, 정답이 있는 74종만으로 평균을 내는지, 아니면 나머지 44종도 0점으로 넣고 118로 나누는지 궁금합니다.

관련하여 아래의 세 가지를 질문드립니다.

1. Public mAP의 클래스 평균은 어떤 목록과 몇 종을 기준으로 계산하나요? Public 정답에 없는 클래스는 평균에서 제외하나요, 아니면 AP를 0점으로 포함하나요?
2. 리더보드의 `sandbox_perfect_submission.csv` (0.58915)는 어떤 제출인가요? 채점 정답을 그대로 제출한 파일인지, 만약 그렇다면 1.00이 나오지 않는 이유가 궁금합니다.
3. 정답에 없는 클래스도 0점으로 포함하는 방식이라면, 이것이 의도된 설계인가요? 또한 Private 평가에도 같은 규칙이 적용되나요?

확인 및 답변 가능한 사항이라면 확인 부탁드려도 괜찮을지 여쭙니다.
감사합니다.



과정 운영 담당_허율 어제 오후 2:42
안녕하세요. 의연 님! 문의주신 내용 답변드립니다~

1. Public mAP의 클래스 평균 기준.
• 평균은 해당 구간(Public)의 정답에 실제로 등장하는 클래스만 대상으로 계산합니다. Public 정답에는 74종이 있고, 이 74종의 AP를 평균합니다..
• 정답에 없는 클래스는 AP를 0점으로 넣지 않고 평균에서 제외합니다. 질문하신 예시로 말씀드리면 118로 나누지 않고 74로 나눕니다..
• 지표는 IoU 0.75~0.95(0.05 간격) 구간의 클래스별 AP를 평균한 mAP입니다..
• 반대로 정답에 있는 클래스를 한 개도 예측하지 않으면 해당 클래스 AP는 0점으로 반영됩니다..

2. sandbox_perfect_submission.csv에 관하여

• 이 파일은 운영팀이 채점 환경을 확인하려고 올려둔 내부 테스트용 제출입니다. 단순 더미데이터이며 참가팀의 성능과 비교하는 기준 점수는 아닙니다..

3. 정답에 없는 클래스의 처리와 Private 평가

• 1번에서 말씀드린 대로 정답에 없는 클래스는 0점으로 포함하지 않고 평균에서 제외합니다..
• Private 평가도 같은 채점 코드로 계산합니다. 클래스 구성도 Public과 동일한 74종이라 같은 기준으로 비교하실 수 있습니다..